

实验十二 循环神经网络

（一）目的和要求

- （1）理解循环神经网络的基本原理和结构。
- （2）掌握使用 Keras 构建循环神经网络的方法。
- （3）了解时间序列数据的预处理方法。
- （4）掌握 LSTM 模型的训练、评估和预测方法。
- （5）掌握模型保存与加载的方法。

（二）实验知识准备

（1）循环神经网络

循环神经网络（Recurrent Neural Network, RNN）是一种具有循环结构的神经网络，能够通过“记忆”前一时刻的信息来处理序列数据。RNN 的关键特性在于其循环连接，即网络中的隐藏层节点不仅接收当前输入，还接收前一个时刻隐藏层的状态。RNN 能够捕捉到数据序列中的时间依赖关系，适用于处理任意长度的输入序列。RNN 在时间维度上共享权重，显著减少了模型参数的数量，有助于提高训练效率。其主要用于自然语言处理（Natural Language Processing, NLP）领域。

（2）自然语言处理

自然语言处理是将自然语言中的句子数值化，包括获取语料、语料预处理和词向量 3 个步骤。其中，获取语料的方式包括整理语料和抓取语料；语料预处理包括数据清洗、分词、词性标注和去停用词等步骤。一般情况下，Keras 提供的数据集已经完成了获取语料和语料预处理两个步骤，故此处不再对这两个步骤进行详细介绍。

（3）词向量

也称为词嵌入，是自然语言处理中语言建模和特征学习技术的统称。它能够将单词嵌入到浮点数的向量空间。它是通过神经网络自行学习创建的，还可以自动创建单词之间上下文的关系，从而形成单词的意义。其中，Word2vec 是一款用于训练词向量的软件工具，它可以将庞大的自然语言文本信息作为学习数据，并将单词之间的物理性距离作为依据进行学习。其中所采用的创建模型方法包括

Skip-gram 和 CBoW。

（4）长短期记忆神经网络

长短期记忆神经网络（long short-term memory, LSTM）是一种特殊的 RNN，能够学习长期依赖信息。它通过引入门控机制（输入门、遗忘门和输出门）来解决传统 RNN 在处理长序列数据时的梯度消失和梯度爆炸问题。LSTM 在时间序列预测、自然语言处理等领域有广泛应用。

（5）序列数据

序列数据是一种有顺序的向量数据，简单地说，序列数据的数据前后具有关联性。例如，“今天阴天，可能要下雨，出门最好带（书包、伞、钥匙）”就是序列数据，人们可以根据前面词语的意思，推测后面横线处最有可能填“伞”。

（6）门控循环单元神经网络

因为长短期记忆神经网络的计算过程较复杂，所以当 LSTM 神经网络中使用较多的记忆单元时，会产生较大的计算量。后来，有学者研究出了门控循环单元神经网络（Gated Recurrent Unit, GRU），它是 LSTM 神经网络的变体，简化了 LSTM 神经网络的计算流程，减少了门控的数量。

（三）实验内容及步骤

（1）假设序列为[[1],[2,3],[4,5,6]]，编写程序实现序列填充，填充方式为'post'，分别显示填充前后序列。示例代码如图 1 所示。

```
1 import tensorflow as tf          #导入所需库
2 s=[[1],[2,3],[4,5,6]]           #初始化填充前序列s
3 print("填充前序列为\n",s)       #显示填充前序列s
4 #填充序列并赋值给a
5 a=tf.keras.preprocessing.sequence.pad_sequences(s,padding='post')
6 print("填充后序列为\n",a)        #显示填充后序列a
```

图 1 序列填充

图 1 所示代码中首先导入了 TensorFlow 库，然后初始化了一个名为 s 的 Python 列表，该列表包含了不同长度的子列表，代表了一个未填充前的序列数据。接着，代码使用 TensorFlow Keras 的 pad_sequences 函数对序列 s 进行了填充操作，填充方式为在序列的尾部添加填充值（默认为 0），以确保所有序列的长度

一致。填充后的序列被赋值给变量 `a`。最后，代码分别打印了填充前后的序列，展示了填充操作的效果。。

(2) 假设序列为`[[1,2,3,9],[4,5,6],[7,8]]`，编写程序实现剪裁序列，最大长度为 2，剪裁方式为'pre'，分别显示剪裁前后序列。示例代码如图 2 所示。

```
1 import tensorflow as tf                #导入所需库
2 s=[[1, 2, 3, 9], [4, 5, 6], [7, 8]]    #初始化剪裁前序列s
3 print("剪裁前序列为\n", s)            #显示剪裁前序列s
4 #剪裁序列并赋值给a
5 a=tf.keras.preprocessing.sequence.pad_sequences(s,maxlen=2)
6 print("剪裁后序列为\n", a)            #显示剪裁后序列a
```

图 2 序列裁剪

图 2 所示代码中使用 TensorFlow Keras 的 `pad_sequences` 函数对一个包含不同长度子列表的序列进行了处理，由于指定的 `maxlen` 参数值小于序列中的最长长度，因此实际上执行了截断操作，从每个序列的开始处删除元素，直到所有序列的长度都等于 `maxlen` 指定的值，最终得到了一个所有序列长度都一致的截断后序列。

(3) 使用 Keras 构建网络模型，包含一个嵌入层，其词汇表的大小为 100，转换后向量的长度为 32，输入序列的长度为 400。显示网络模型参数信息、`embeddings` 矩阵的值及输出向量的形状。示例代码如图 3。

```

1 import tensorflow as tf          #导入所需库
2 import numpy as np
3 model=tf.keras.Sequential()      #构建空的网络模型model
4 #创建嵌入层
5 embedding=tf.keras.layers.Embedding(output_dim=32, input_dim=100, input_length=400)
6 model.add(embedding)             #添加到网络模型model中
7 model.summary()                  #显示网络模型的参数信息
8 #显示embeddings矩阵的值
9 print("embeddings矩阵=\n", embedding.get_weights())
10 text="Deep learning is an important concept raised by the current sciences."
11 #定义分词器对象
12 token=tf.keras.preprocessing.text.Tokenizer(num_words=100)
13 token.fit_on_texts(text)         #分词
14 input=token.texts_to_sequences(text) #输出向量序列
15 #序列填充
16 test_seq=tf.keras.preprocessing.sequence.pad_sequences(input,
17 padding='post', maxlen=400, truncating='post')
18 #使用向量序列应用网络模型
19 output_array=model.predict(test_seq)
20 #显示输出向量的形状
21 print("输出矩阵的形状=", output_array.shape)

```

图 3 keras 搭建模型

图 4 所示代码构建一个能够处理文本数据的神经网络模型。该模型的核心是一个嵌入层（Embedding layer），它负责将文本中的每个词汇索引转换成 32 维的密集向量，以便神经网络能够更有效地处理。为了构建这个嵌入层，代码设定了词汇表的大小为 100，即只考虑文本中最常见的 100 个单词（分词器会考虑文本中最常见的 100 个单词，并为它们分配索引。这些单词及其对应的索引就构成了词汇表），同时规定了输入序列的长度为 400，确保所有输入数据具有统一的维度。在模型构建过程中，利用了文本分词器对文本进行了预处理，将其转换为整数序列。为了适配神经网络的输入要求，对序列进行了填充操作，使其长度达到规定的 400。

（4）使用 Keras 构建简单循环神经网络，其中嵌入层词汇表的大小为 10，转换后向量的长度为 5，输入序列的长度为 6；简单循环层有 128 个结点，使用 Tanh 函数作为激活函数；输出层有 5 个结点，使用 Softmax 函数作为激活函数。显示网络模型参数信息。示例代码如图 4。

```

1 import tensorflow as tf          #导入所需库
2 model=tf.keras.Sequential()      #创建空的网络模型model
3 #创建嵌入层并添加到model中
4 model.add(tf.keras.layers.Embedding(10,5,input_length=6))
5 #创建简单循环层并添加到model中
6 model.add(tf.keras.layers.SimpleRNN(128))
7 #创建全连接层作为输出层，并添加到model中
8 model.add(tf.keras.layers.Dense(5,activation='softmax'))
9 model.summary()                  #显示网络模型的参数信息

```

图 4 构建 RNN 网络模型

图 4 所示代码创建了一个空的神经网络模型，并向模型中添加了一个嵌入层，该层负责将整数索引（代表词汇表中的单词）转换为固定大小的密集向量，其中词汇表大小为 10，嵌入向量的长度为 5，且每个输入序列的长度被固定为 6。之后，添加了一个简单循环层（SimpleRNN），用于处理序列数据并捕捉其中的时序依赖关系。最后，添加了一个全连接层作为输出层，该层包含 5 个神经元，并使用 softmax 激活函数进行多分类预测。通过调用 model.summary() 方法，代码展示了网络模型的参数信息，包括各层的名称、类型、输出形状和参数数量等。

（5）使用 Keras 构建 LSTM 神经网络，其中嵌入层词汇表的大小为 10，转换后向量的长度为 5，输入序列的长度为 6；LSTM 神经网络的记忆层有 128 个结点，使用 Tanh 函数作为激活函数；输出层有 5 个结点，使用 Softmax 函数作为激活函数。示例代码如图 5。

```

1 import tensorflow as tf          #导入所需库
2 model=tf.keras.Sequential()      #创建空的网络模型model
3 #创建嵌入层并添加到model中
4 model.add(tf.keras.layers.Embedding(10,5,input_length=6))
5 #创建长短期记忆层并添加到model中
6 model.add(tf.keras.layers.LSTM(128))
7 #创建全连接层作为输出层，并添加到model中
8 model.add(tf.keras.layers.Dense(5,activation='softmax'))
9 model.summary()                  #显示网络模型的参数信息

```

图 5 构建 LSTM 网络

图 5 所示代码用于构建和训练神经网络。接着，创建了一个空的顺序模型（`Sequential model`），这是 TensorFlow Keras 中用于线性堆叠网络层的模型。然后，向模型中添加了一个嵌入层，该层负责将整数索引（代表词汇表中的单词，词汇表大小为 10）转换为 5 维的密集向量，并且指定了输入序列的长度为 6。之后，添加了一个长短期记忆层，其单元数为 128。最后，添加了一个全连接层作为输出层，该层包含 5 个神经元，并使用 `softmax` 激活函数进行多分类预测。通过调用 `model.summary()` 方法，代码展示了网络模型的参数信息，包括各层的名称、输出形状和参数数量等，这对于理解模型结构和进行模型调试非常有帮助。

(6) 实训项目

1. IMDb 数据集

IMDb 数据集来源于全球最大的电影数据库网站——IMDb（互联网电影数据库）。该数据集包含了大量的电影评论，通常用于情感分析和文本分类任务。具体来说，IMDb 数据集通常包含 50,000 条电影评论，其中 25,000 条用于训练，另外 25,000 条用于测试。每条评论都已明确标记为正面（好评）或负面（差评），基于 10 分制评分系统简化为二分类问题。

2. 基于 IMDb 数据集，使用 Keras 构建 GRU 神经网络模型，并对网络模型进行编译、训练和评估。该实验应包含以下步骤：

(1) 准备数据。

- ① 导入本项目所用到的库，包括 TensorFlow、NumPy 和 Matplotlib 等。
- ② 从 Keras 中导入 IMDb 数据集。
- ③ 对训练集和测试集的特征值分别进行序列填充。

(2) 构建 GRU 神经网络模型。

- ① 构建空的顺序模型。
- ② 为网络模型添加嵌入层，词汇表的大小为 4000，转换后向量的长度为 32，输入序列的长度为 400。
- ③ 为网络模型添加 Dropout 层，丢弃概率为 0.3。
- ④ 为网络模型添加 GRU 层，输出维度为 64。
- ⑤ 为网络模型添加 Dropout 层，丢弃概率为 0.3。
- ⑥ 为网络模型添加隐藏层作为输出层，结点个数为 1，使用 Sigmoid 函数作为激活函数。
- ⑦ 使用 `summary()` 函数显示模型各层的参数信息。

(3) 编译、训练和评估 GRU 神经网络模型。

- ① 编译 GRU 神经网络模型，其中，优化器使用 RMSprop 优化器，损失函数使用二元交叉熵损失函数，性能评估函数使用准确率函数。
- ② 训练 GRU 神经网络模型，其中，设置批量的大小为 64，迭代次数为 10，验证集的数据占比为 0.2。
- ③ 使用测试集数据对 GRU 神经网络模型进行评估，其中，设置批量的大小

为 64，日志显示模式为 2。

(4) 创建子图，在子图 1 中分别绘制训练集和验证集损失函数值的折线图；在子图 2 中分别绘制训练集和验证集准确率的折线图。

3. 根据上一步所示的流程，将模型替换为 LSTM 神经网络重新进行训练，并比较 GRU 与 LSTM 的区别。